

HENRY PÉREZ GARCÍA

1. Optimización en sistemas financieros:

a. Imagina que trabajas en una aplicación bancaria que debe procesar transacciones en tiempo real. ¿Qué estrategias aplicarías para mejorar el rendimiento y la escalabilidad? Considera aspectos como concurrencia, caché y patrones de diseño.

Usaría procesamiento asíncrono y colas (RabbitMQ/Kafka) para manejar muchas transacciones simultáneas. Implementaría caché con Redis para consultas frecuentes como saldos o configuraciones. También aplicaría balanceo de carga y microservicios para escalar horizontalmente.

2. Seguridad en APIs financieras:

a. Explica cómo protegerías una API que maneja información sensible de cuentas

bancarias contra ataques como inyección SQL, CSRF, XSS y otros ataques comunes.

Usaría consultas parametrizadas o JPA para evitar SQL Injection.

Implementaría autenticación y autorización con JWT/OAuth2.

Activaría HTTPS para cifrar la comunicación y validaría y sanitizaría todos los datos de entrada, también configuraría protección CSRF en aplicaciones con sesión

Adicional agregaría rate limiting y logs de auditoría para detectar ataques

3. Transacciones en sistemas distribuidos:

a. En un sistema bancario distribuido, ¿cómo implementarías la consistencia y el

manejo de errores en una API que procesa transferencias entre cuentas en diferentes servicios?

Implementaría eventos y mensajería para comunicación confiable.

Aplicaría rollback o compensaciones si una operación falla.

Usaría idempotencia para evitar transferencias duplicadas y registraría logs y trazabilidad para auditoría y recuperación.

4. Pruebas unitarias y de integración:

a. Describe cómo diseñarías una suite de pruebas que asegure la correcta operación de una API bancaria, considerando tanto pruebas unitarias como de integración. ¿Qué herramientas utilizarías?

Haría pruebas unitarias para validar lógica de negocio aislada, pruebas de integración para validar base de datos, APIs y servicios externos.

Utilizaría herramientas como JUnit, Mockito, Postman

5. Front-end:

a. En una aplicación bancaria que muestra el saldo de las cuentas, ¿cómo gestionarías el estado y la autenticación en el front-end, garantizando la seguridad y la coherencia de los datos?

Gestionaría el estado con Redux.

Almacenaría tokens de forma segura, preferiblemente en cookies HttpOnly y validaría la expiración de sesión y renovarían tokens automáticamente.

6. SPRING BOOT

a. Que pasa en una aplicación internamente cuando usas la anotación @SpringBootApplication y cómo afecta el arranque de una aplicación?

Combina @Configuration, @EnableAutoConfiguration y @ComponentScan.

Spring escanea componentes automáticamente y configura beans necesarios

Simplifica y acelera el arranque de la aplicación.

b. ¿Cómo funciona el ciclo de vida de un beans en Spring y como podrías intervenir en él?

Spring crea el bean, inyecta dependencias e inicializa el objeto, luego el bean queda disponible para la aplicación, al finalizar, Spring destruye el bean.

Se puede intervenir usando @PostConstruct, @PreDestroy

c. ¿Cómo personalizarías el comportamiento de la auto-configuración en Spring

Boot sin romper la filosofía de 'convención sobre configuración'?

Usaría propiedades en application.properties o application.yml.

Crearía beans personalizados para sobrescribir configuraciones por defecto.